WISDOM
APP

Claude Code Master Build Prompt · v2.0

Next.js 14 · FastAPI · PostgreSQL · RBAC + ABAC · Solo Dev Edition

The Ark Centre · Santhosa Samsara · Bengaluru, Karnataka

Phased · File-level deliverables · Checkpoint-gated · Jerry Nixon architecture principles applied

March 2026

1. Architecture Philosophy & Stack Decision

This document is the complete Claude Code build prompt for WISDOM APP. Before writing a single line of code, these architecture decisions were made deliberately based on three inputs: the WISDOM feature specification, Jerry Nixon's Modern Architecture 101 principles ('Simplicity is the best architecture — always defer decisions'), and the reality that this is a solo developer build with AI assistance.

1.1 The Solo Dev Rule

Jerry Nixon's full enterprise architecture — Kong APIM, Celery workers, CQRS replicas, Service Bus, Event Hub — is designed for teams. For a solo build, each additional system is a system you alone must learn, configure, debug, and maintain. The correct approach is:

- Build a domain-structured monolith with clean internal seams
- Use managed cloud services to eliminate DevOps overhead
- Add complexity only when user count or team size demands it
- Design the schema and folder structure so services can be extracted later with minimal refactoring

1.2 Architecture Principles Applied (from Jerry Nixon)

Jerry Nixon Principle	How WISDOM Implements It
Client never talks to DB directly	Next.js → FastAPI → PostgreSQL. DB is never exposed.
API decoupled from Client	Two separate repos. Contract (OpenAPI schema) is the shared boundary.
Simplicity first	Monolith FastAPI app, domain-structured. No microservices in v1.
Defer decisions	Service Bus, CQRS replicas, Event Hub deferred until scale demands them.
OpenTelemetry from Day 1	Sentry in both Next.js and FastAPI from Phase 1. Never added last.
Cache aggressively	Redis L2 cache on all read-heavy endpoints. Assessment results, room availability, public articles.
Async for external calls	WhatsApp, Razorpay webhooks, AI features all use FastAPI BackgroundTasks.
CDN for public portal	Cloudflare free tier — Indian parents on mobile feel the difference.
Seams built in	app/cases/, app/assessments/ etc. Each domain folder can be lifted into a microservice later.
Offline-first client	Next.js PWA with Workbox — parents filling DSM-5 assessments on poor connectivity keep their progress.

1.3 Why Next.js + FastAPI (not Reflex, not pure Node.js)

Option	Pros	Cons for WISDOM
Reflex (original)	Python everywhere	Tiny ecosystem, no mobile, no SEO, tiny talent pool in India, generates JS you don't control
Pure Next.js + Node.js	One language, huge talent pool	AI/ML libraries are Python-native. Awkward to port LiteLLM scoring logic to TypeScript.
Next.js + FastAPI ✓	Best frontend framework + Python AI/ML stays natural. Huge talent pool in Bengaluru for both.	Two repos to manage. Acceptable for solo dev with AI assistance.

1.4 Future ML/AI Scalability Path

The FastAPI backend is structured so the app/ai/ module evolves naturally:

- v1: LiteLLM calls for note summarisation, parent advice, PDF→Survey parsing
- v2: DSM-5 scoring model, behavioural trend detection — Python scikit-learn or PyTorch, same repo
- v3: Full ML inference service extracted from app/ai/ into its own container — zero schema changes, same PostgreSQL
- v4: Data lake (event-driven via PostgreSQL LISTEN/NOTIFY → serverless function) for long-term outcome analytics

Because AI lives in Python from day one, none of these steps require touching Next.js or rewriting the database schema.

2. Final Tech Stack

Layer	Technology	Rationale
Frontend	Next.js 14 (App Router)	*SSR for public portal SEO, RSC for performance, PWA-ready*
UI Components	Tailwind CSS v4 + shadcn/ui	*Production-grade accessible components, not raw Tailwind*
Animations	Framer Motion	*Smooth transitions for the child-facing survey UI*
Backend API	FastAPI (Python 3.11)	*Async-native, auto OpenAPI docs, perfect for AI/ML*
ORM	SQLAlchemy 2.0 (async) + Alembic	*Type-safe, async DB access, migration history*
Database	PostgreSQL 16 (Supabase hosted)	*Managed, free tier, built-in auth helpers, Row Level Security*
Cache	Redis 7 (Upstash hosted)	*Managed free tier, no Redis server to maintain solo*
Auth	NextAuth.js (frontend) + JWT (API)	*Separate auth flows for staff vs parent vs public*
Admin Panel	Custom Next.js + RBAC+ABAC engine	*Full access control UI (see Section 4)*
Async Tasks	FastAPI BackgroundTasks → Celery	*Start simple. Add Celery only when tasks timeout at scale*
AI Gateway	LiteLLM	*Model-agnostic. Swap GPT-4o/Claude/Gemini without code changes*
PDF Parsing	pdf-parse + PyMuPDF	*DSM-5 and other assessment PDFs*
Messaging	Meta WhatsApp Cloud API (async)	*Appointment reminders, risk alerts, campaigns*
Payments	Razorpay + Stripe (international)	*UPI for India, Stripe for foreign donors*
Observability	Sentry (Day 1) → OpenTelemetry	*Jerry Nixon: think about telemetry upfront, never last*
CDN	Cloudflare (free tier)	*Public portal assets — Indian mobile users feel the difference*
Containers	Docker Compose (dev)	*Consistent local environment*
Hosting	Railway or Render	*Git-push deployment, managed SSL, no solo DevOps overhead*
CI/CD	GitHub Actions	*Lint, test, build on every PR*

Layer	Technology	Rationale
Source Control	GitHub (private)	*Standard, free, integrates with all tools above*

3. Repository & Folder Structure

Two repositories, one monorepo folder for local development. The contract between them is the FastAPI OpenAPI schema — this never changes without a version bump.

3.1 wisdom-frontend/ (Next.js 14)

```
wisdom-frontend/
  app/                          ← App Router root
    (auth)/login/               ← Staff login page
    (auth)/parent-login/        ← Parent login (separate flow)
    (staff)/dashboard/          ← Therapist & staff portal
    (staff)/cases/              ← Case records CRUD
    (staff)/assessments/        ← Assessment management
    (staff)/scheduling/         ← Room scheduling calendar
    (staff)/analytics/          ← Analytics dashboard
    (parent)/portal/            ← Parent/caretaker portal
    (parent)/assessments/       ← Parent-facing survey forms
    (public)/                   ← Public access platform
    admin/                      ← Admin panel (RBAC+ABAC UI)
    admin/users/                ← User management
    admin/roles/                ← Role & permission editor
    admin/audit-log/            ← Audit trail viewer
  components/
    ui/                         ← shadcn/ui components
    forms/                      ← Assessment survey renderer
    charts/                     ← Score trend graphs
    admin/                      ← Admin panel components
  lib/
    api.ts                      ← FastAPI client (typed)
    auth.ts                     ← NextAuth config
    rbac.ts                     ← Client-side permission checks
  middleware.ts                  ← Route guards (RBAC enforcement)
```

3.2 wisdom-backend/ (FastAPI)

```
wisdom-backend/
  app/
    main.py                     ← FastAPI app entry point
    config.py                   ← Settings from environment
    database.py                 ← Async SQLAlchemy session
    redis.py                    ← Redis client
    sentry.py                   ← Sentry init (Day 1)
    |
    cases/                      ← Child case management
    |   router.py
    |   service.py              ← All business logic
    |   models.py               ← SQLAlchemy models
    |   schemas.py              ← Pydantic request/response
    |
    assessments/                ← DSM-5 + other assessments
```

```
  scheduling/                ← Room booking & calendars
  messaging/                 ← WhatsApp (async)
  payments/                  ← Razorpay + Stripe
  ai/                        ← LiteLLM features
  analytics/                 ← Aggregation endpoints
  public/                    ← Public portal content
  donations/                 ← Crowdfunding + recurring
  |
  auth/
  |   jwt.py                 ← Token creation & validation
  |   rbac.py                ← RBAC engine
  |   abac.py                ← ABAC engine
  |   guards.py              ← FastAPI dependency guards
  |
  admin/
      router.py              ← Admin API endpoints
      user_service.py        ← User CRUD + role assignment
      audit_service.py       ← Audit log queries
alembic/
seeds/
  rooms.py                   ← Ark Centre room inventory
  dsm5.py                    ← DSM-5 Level 1 assessment
tests/
Dockerfile
docker-compose.yml
.env.example
```

4. Access Control: RBAC + ABAC Hybrid Model

This is the most critical non-clinical system in WISDOM. It determines who can see what child data, who can trigger risk alerts, and who can access the admin panel. Getting this wrong has legal and safeguarding consequences.

4.1 Core Concept

Why RBAC + ABAC Hybrid — not just RBAC

- Pure RBAC (Role-Based Access Control) answers: 'Can a Therapist read case records?' — Yes or No based on role.
- ABAC (Attribute-Based Access Control) answers: 'Can THIS Therapist read THIS child's record?' — based on the relationship between the user and the resource.
- WISDOM needs both. A Therapist can read case records (RBAC) — but only for children assigned to them (ABAC).
- A Supervisor can read any case in their team (RBAC + ABAC team scope).
- An Admin can read everything (RBAC overrides ABAC).
- Open source implementation: Casbin (Python) for the policy engine. Zero licence cost.

4.2 The 7 Roles

Role	Scope	Key Permissions
Super Admin	Full system	All permissions. User management, role editor, audit log, system config. Only 1–2 accounts.
Admin	Organisation	User management, all case records, analytics, invoicing. Cannot edit system config.
Chief Therapist	Organisation	All case records, assessment uploads, assign therapists to children, view all analytics.
Supervisor	Team scope	Case records for their team only, therapist notes for their team, cannot see other teams.
Therapist	Assigned cases	Case records for assigned children only, session notes, assessment results, progress reports.
Nurturer	Assigned cases	View case records for assigned children, add observations, cannot edit clinical assessments.
Staff	Operational	Room scheduling, invoicing, WhatsApp campaigns, no access to clinical case records.

4.3 ABAC Attribute Rules

These rules run AFTER the RBAC check. Even if the role permits an action, the ABAC check can deny it based on the relationship between the user and the resource:

Resource	ABAC Rule	Overridden By
Child case record	*user.id must be in case.assigned_staff[]*	Admin, Chief Therapist (see all)
Session notes	*user.id must be the note author OR case supervisor*	Admin, Chief Therapist
Assessment results	*user.id must be assigned therapist or supervisor*	Admin, Chief Therapist
Room bookings	*user.id must be the booking creator OR admin*	Admin, Staff (full access)
Invoice records	*Staff role required + is_billing_staff attribute*	Admin
WhatsApp campaigns	*Staff role required + is_comms_staff attribute*	Admin
Audit log	*Super Admin only — no ABAC override*	None — immutable restriction

4.4 Admin Panel — UI Features

The Admin Panel lives at /admin in the Next.js app. It is server-side rendered with route guards enforced in middleware.ts. Only Super Admin and Admin roles can access it.

Admin Panel Section	Capabilities
User Management	Create/edit/deactivate users. Assign roles. Reset passwords. View last login. Bulk import via CSV.
Role Editor	View all 7 roles and their permission sets. Customise per-role permissions (Casbin policy editor UI). Cannot remove safety-critical permissions (suicidal ideation alert).
ABAC Attribute Manager	Assign users to teams, assign therapists to children, set billing/comms staff attributes.
Consent Tracker	View consent status for every child record. Flag missing consents. Export for DPDP Act compliance.
Audit Log Viewer	Immutable log of all reads/writes to child records. Filter by user, date, action. Export to CSV. Super Admin only.
Session Monitor	Active user sessions, force-logout capability, suspicious login alerts.
System Config	WhatsApp API credentials, LiteLLM model selection, Razorpay keys (Super Admin only). All stored encrypted.
Assessment Library	Upload new assessment PDFs for AI parsing. Review parsed survey before publishing. Assign to parent groups.

4.5 Open Source Tools for RBAC+ABAC

Tool	Role in WISDOM
Casbin (Python)	Policy engine for RBAC+ABAC rules. Stores policies in PostgreSQL. Mature, battle-tested, MIT licence.
pycasbin FastAPI middleware	Integrates Casbin as a FastAPI dependency. Automatic enforcement on every route.
NextAuth.js	Session management + JWT on the frontend. Handles staff vs parent vs public auth flows.
Python-Jose	JWT creation and validation in FastAPI.
Argon2 (via passlib)	Password hashing. Stronger than bcrypt, recommended for new projects.

5. How to Use This Prompt in Claude Code

1	Open Claude Code in an empty project folder: mkdir wisdom && cd wisdom
2	Attach all three PDF files before sending any message: • WISDOM_APP.pdf • APA-DSM5TR-Level1MeasureChildAge11To17.pdf • New_The_Ark_Proposal_-Rooms.pdf
3	Paste the Global System Context block (Section 6) as your very first message. Wait for confirmation.
4	Then paste the Phase 1 prompt (Section 7). Complete all deliverables and the checkpoint before Phase 2.
5	Repeat for each phase in order. Never skip a checkpoint — they exist to catch mistakes early.
6	When Claude Code makes an architectural decision not in this document, it must ask you first and log the decision in docs/adr/ (Architecture Decision Records).

6. Global System Context Block

Copy everything inside the box below and paste it as your very first message to Claude Code. Attach all three PDFs before sending.

— PASTE START —

```
You are a Senior Python Engineer, Next.js Developer, and Health Platform
Architect.
You are building WISDOM APP — a production-grade platform for The Ark Centre
(Santhosa Samsara, Bengaluru, India), a trauma-recovery facility for children.

THREE REFERENCE DOCUMENTS ARE ATTACHED — read them all before responding:
  1. WISDOM_APP.pdf              — full feature spec by George Ebenezer
  2. APA-DSM5TR-...pdf           — first clinical assessment to digitize
  3. New_The_Ark_Proposal_Rooms  — physical facility with all room names

TECH STACK (non-negotiable):
  Frontend  : Next.js 14 App Router + Tailwind CSS v4 + shadcn/ui + Framer Motion
  Backend   : FastAPI (Python 3.11) — domain-structured monolith
  ORM       : SQLAlchemy 2.0 (async) + Alembic migrations
  Database  : PostgreSQL 16 (Supabase for managed hosting)
  Cache     : Redis 7 (Upstash for managed hosting)
  Auth      : NextAuth.js (frontend) + JWT + Casbin RBAC+ABAC (backend)
  Admin     : Custom Next.js admin panel at /admin with full user/role management
  AI        : LiteLLM gateway
  Messaging : Meta WhatsApp Cloud API (async via BackgroundTasks)
  Payments  : Razorpay (primary) + Stripe (international)
  Monitor   : Sentry installed Day 1 in both repos
  CDN       : Cloudflare (public portal static assets)
  Hosting   : Railway or Render (git-push deployment, no solo DevOps overhead)

ACCESS CONTROL: RBAC + ABAC Hybrid using Casbin.
  7 Roles: Super Admin, Admin, Chief Therapist, Supervisor,
           Therapist, Nurturer, Staff
  ABAC rule: even if role permits, the user must have a direct
  assignment relationship with the child record to access it.

WORKING RULES:
  1. Always create REAL files — never just describe what a file should contain.
  2. Follow the phase structure exactly. Do not jump ahead.
  3. At every CHECKPOINT: list every file you created, then STOP and wait.
  4. Any architectural decision not covered here: ask me before deciding.
  5. Never hardcode secrets. All credentials via environment variables.
  6. Sentry must be initialised in both repos before any feature code is written.
  7. SAFETY P0: DSM-5 Domain XII (Suicidal Ideation) = any 'Yes' answer must
     trigger an IMMEDIATE in-app alert + WhatsApp message to the assigned
     therapist. This is non-negotiable and cannot be deferred.
  8. All child PII encrypted AES-256 at rest. TLS 1.3 in transit.
  9. Audit log: every read/write to a child record is immutably logged.
 10. Consent: parent consent must be recorded before any child record is created.

Confirm you have read all three PDFs. Then wait for my Phase 1 prompt.
```

— PASTE END —

7. Build Phases

Six phases. Each phase ends with a mandatory checkpoint. Do not proceed until the checkpoint is confirmed. The order is deliberate — each phase builds on the last.

PHASE 1 Project Scaffold, Database & Observability · *Week 1–2*

Goals for this phase

- Initialise both repos (wisdom-frontend, wisdom-backend) with correct folder structure
- Configure FastAPI app with all domain folders as empty modules
- Define all PostgreSQL models (SQLAlchemy 2.0 async)
- Create Alembic migration 001_initial — run and confirm 0 errors
- Configure Docker Compose with PostgreSQL 16 + Redis 7
- Seed the database: all Ark Centre rooms from the Rooms PDF
- Seed the database: full DSM-5 Level 1 assessment from the APA PDF
- Install Sentry in both repos — THIS MUST HAPPEN IN PHASE 1, NOT LATER
- Configure .env.example with every environment variable documented

Deliverables — Claude Code must create all files listed below

File / Deliverable	Description	Type
wisdom-backend/app/main.py	FastAPI app init, CORS, Sentry, router includes	Python
wisdom-backend/app/config.py	Pydantic Settings — all env vars typed	Python
wisdom-backend/app/database.py	Async SQLAlchemy engine + session	Python
wisdom-backend/app/sentry.py	Sentry SDK init — Phase 1, not Phase 6	Python
wisdom-backend/app/cases/models.py	Child/Client case models	SQLAlchemy
wisdom-backend/app/auth/models.py	User, Role, Permission, UserAttribute models	SQLAlchemy
wisdom-backend/app/assessments/models.py	Assessment, Section, Question, Domain, Score models	SQLAlchemy
wisdom-backend/app/scheduling/models.py	Room, Booking, Equipment models	SQLAlchemy
wisdom-backend/app/payments/models.py	Invoice, Payment, Donation models	SQLAlchemy
alembic/versions/001_initial.py	All tables — run and confirm clean	Alembic
seeds/rooms.py	Seed all Ark rooms (exact names from PDF)	Python
seeds/dsm5.py	Seed DSM-5 L1: 25 questions, 12 domains, thresholds, Domain XII flag	Python
docker-compose.yml	PostgreSQL 16 + Redis 7 + FastAPI	Docker

File / Deliverable	Description	Type
.env.example	Every env var documented with description	Config
wisdom-frontend/ (scaffolded)	Next.js 14 App Router, Tailwind, shadcn/ui, Sentry installed	Next.js

🛑 **CHECKPOINT — Stop and confirm before continuing**

- ☐ Show me: tree wisdom-backend/app (all domain folders present)
- ☐ Show me: alembic current — migration applied, 0 errors
- ☐ Confirm: all 12 DSM-5 domains and 25 questions seeded correctly
- ☐ Confirm: all Ark rooms seeded (6× Counselling, 2× Training, Aqua Pool, etc.)
- ☐ Confirm: Sentry is initialised in both repos and a test event reaches the dashboard
- ☐ Run: docker-compose up — PostgreSQL and Redis healthy

PHASE 2 Authentication, RBAC+ABAC & Admin Panel

Week 3–4

Goals for this phase

- Implement JWT authentication (access + refresh tokens, httpOnly cookies)
- Implement Casbin policy engine with all 7 roles and ABAC attribute rules
- Build FastAPI route guards using Casbin as a dependency
- Create separate auth flows: staff login, parent/caretaker login, public (no auth)
- Build the full Admin Panel in Next.js: user management, role editor, ABAC attribute manager, audit log viewer, consent tracker, session monitor
- Write pytest tests covering every role × resource × ABAC combination
- Enforce route guards in Next.js middleware.ts for all protected routes

Deliverables — Claude Code must create all files listed below

File / Deliverable	Description	Type
wisdom-backend/app/auth/jwt.py	JWT creation, validation, refresh token rotation	Python
wisdom-backend/app/auth/rbac.py	Casbin RBAC model + policy loader from PostgreSQL	Python
wisdom-backend/app/auth/abac.py	ABAC attribute rules (assignment checks)	Python
wisdom-backend/app/auth/guards.py	FastAPI Depends() guards for all 7 roles + ABAC	Python
wisdom-backend/app/admin/router.py	Admin API: user CRUD, role assignment, audit log	Python
wisdom-backend/app/admin/user_service.py	User management business logic	Python
wisdom-backend/app/admin/audit_service.py	Immutable audit log read/write	Python
wisdom-frontend/app/admin/	Full admin panel UI (all 8 sections from Section 4.4)	Next.js
wisdom-frontend/middleware.ts	Route guards — staff, parent, public, admin routes	Next.js
wisdom-frontend/lib/rbac.ts	Client-side permission checks (mirrors backend roles)	TypeScript
docs/rbac_matrix.md	Complete role × action permission matrix	Markdown
docs/abac_rules.md	ABAC attribute rules documented	Markdown
tests/test_auth.py	All role × resource × ABAC combinations	pytest

🛑 **CHECKPOINT — Stop and confirm before continuing**

- ☐ Show me: docs/rbac_matrix.md — all 7 roles, all actions
- ☐ Demonstrate: a Therapist CANNOT access a case not assigned to them (ABAC block)
- ☐ Demonstrate: a Super Admin CAN access the audit log; a Therapist cannot
- ☐ Show me: the Admin Panel user management page in the browser
- ☐ Show me: role editor — can add/remove permissions from a role
- ☐ All auth tests pass: pytest tests/test_auth.py -v

PHASE 3 Core Clinical Modules & Assessment Engine

Week 5–7

Goals for this phase

- Build child/client case record (full CRUD with ABAC enforcement)
- Build therapy session logging module
- Build intervention planning module
- Build the PDF→Survey pipeline using PyMuPDF + LiteLLM
- Digitize the attached DSM-5 Level 1 PDF as the first live assessment — use the actual PDF, do not mock
- Build the scoring engine: domain scores, risk thresholds, Domain XII suicidal ideation P0 alert
- Build parent-facing survey UI in Next.js (mobile-first, Framer Motion, save-and-resume)
- Add assessment results to the child case record with score trend graphs

Deliverables — Claude Code must create all files listed below

File / Deliverable	Description	Type
wisdom-backend/app/cases/router.py	Case CRUD endpoints with ABAC guards	Python
wisdom-backend/app/cases/service.py	Case business logic	Python
wisdom-backend/app/assessments/pdf_parser.py	PyMuPDF text extraction from uploaded PDFs	Python
wisdom-backend/app/assessments/ai_parser.py	LiteLLM: PDF text → structured survey JSON	Python
wisdom-backend/app/assessments/scoring.py	Domain scores, thresholds, Domain XII P0 flag	Python
wisdom-backend/app/assessments/router.py	Assessment CRUD + submission endpoints	Python
wisdom-frontend/app/(staff)/cases/	Case record list, detail, create, edit pages	Next.js
wisdom-frontend/app/(parent)/assessments/	Mobile-first survey renderer with save+resume	Next.js
wisdom-frontend/components/charts/	Score trend graphs (Recharts or Tremor)	Next.js
tests/test_scoring.py	DSM-5 domain scoring, Domain XII P0 alert trigger	pytest

CHECKPOINT — Stop and confirm before continuing

- ☐ Upload the attached DSM-5 PDF through the admin Assessment Library — confirm it parses to a correctly structured survey (25 questions, 12 domains)
- ☐ Submit a test response with Domain XII (Suicidal Ideation) = Yes — confirm a P0 risk alert is generated and logged

- ☐ Show me a rendered child case record with at least one assessment result and a score trend graph
- ☐ Show me the parent-facing survey on a mobile viewport (375px width)
- ☐ All scoring tests pass: pytest tests/test_scoring.py -v

PHASE 4 Room Scheduling, WhatsApp & Payments · *Week 8–9*

Goals for this phase

- Build the room scheduling system: calendar UI, conflict detection, recurring sessions, maintenance windows
- Integrate Meta WhatsApp Cloud API using FastAPI BackgroundTasks (never synchronous)
- Build notification service: appointment reminders, assessment due reminders, risk alert routing
- Build invoice generation for sessions, workshops, room rentals, donations
- Integrate Razorpay (UPI + card) and Stripe (international donors)
- Build the donation portal: one-time, recurring, crowdfunding campaigns, donor recognition
- All external API calls (WhatsApp, Razorpay, Stripe) must have dead-letter logging — if a call fails 3 times, log to dead_letter_queue table

Deliverables — Claude Code must create all files listed below

File / Deliverable	Description	Type
wisdom-backend/app/scheduling/router.py	Room booking API + conflict detection	Python
wisdom-backend/app/scheduling/service.py	Recurring sessions, maintenance windows	Python
wisdom-backend/app/messaging/whatsapp.py	WhatsApp Cloud API wrapper	Python
wisdom-backend/app/messaging/notifications.py	Reminder scheduler (BackgroundTasks)	Python
wisdom-backend/app/messaging/dead_letter.py	Failed message queue + retry logging	Python
wisdom-backend/app/payments/razorpay.py	Razorpay integration + webhook handler	Python
wisdom-backend/app/payments/stripe.py	Stripe integration + webhook handler	Python
wisdom-backend/app/payments/invoicing.py	Invoice generator (PDF output)	Python
wisdom-backend/app/donations/router.py	Donation portal API	Python
wisdom-frontend/app/(staff)/scheduling/	Room calendar UI (FullCalendar or similar)	Next.js
wisdom-frontend/app/(public)/donate/	Donation portal (public-facing)	Next.js
tests/test_scheduling.py	Conflict detection, recurring sessions	pytest
tests/test_payments.py	Razorpay + Stripe webhook validation	pytest

🛑 **CHECKPOINT — Stop and confirm before continuing**

- ☐ Book two sessions in the same room at the same time — confirm conflict is detected and blocked
- ☐ Send a test WhatsApp appointment reminder to a test number
- ☐ Simulate a WhatsApp API failure 3 times — confirm the message appears in dead_letter_queue
- ☐ Generate a PDF invoice for a therapy session
- ☐ Complete a test Razorpay UPI payment in sandbox mode
- ☐ Create a donation campaign and process a recurring donation

PHASE 5 Public Portal, Analytics & AI Features · *Week 10–11*

Goals for this phase

- Build the public access platform: articles, resource directory, community forums, counselor matching, crisis support, workshops
- Build the analytics dashboard for staff: therapy trends, assessment score trends, room utilisation, donation analytics
- Implement all LiteLLM AI features: therapy note summarisation, parent advice generation, behavioural risk detection, clinical summary reports
- Build WhatsApp marketing flows: workshop invitations, donation campaigns
- Add Cloudflare CDN for all public portal static assets
- Implement PWA offline capability (Workbox service worker) for the parent portal

Deliverables — Claude Code must create all files listed below

File / Deliverable	Description	Type
wisdom-backend/app/public/router.py	Public portal content API	Python
wisdom-backend/app/analytics/router.py	Aggregation endpoints for dashboard	Python
wisdom-backend/app/ai/summariser.py	Therapy note summarisation (LiteLLM)	Python
wisdom-backend/app/ai/advice.py	Parent advice generation (LiteLLM)	Python
wisdom-backend/app/ai/risk_detector.py	Behavioural risk detection (LiteLLM)	Python
wisdom-backend/app/ai/clinical_summary.py	Clinical summary reports (LiteLLM)	Python
wisdom-frontend/app/(public)/	Full public portal (all pages)	Next.js
wisdom-frontend/app/(staff)/analytics/	Analytics dashboard with charts	Next.js
wisdom-frontend/public/sw.js	Workbox service worker — offline PWA	JavaScript
tests/test_ai.py	AI service unit tests with mocked LiteLLM	pytest

CHECKPOINT — Stop and confirm before continuing

- ☐ Show me the public portal — confirm it is accessible without authentication
- ☐ Show me the analytics dashboard with sample data for at least 4 metrics
- ☐ Run AI note summarisation on a sample therapy note — show me the output
- ☐ Confirm the parent portal works offline: disconnect network, fill a survey, reconnect, confirm data syncs

- [ ] Send a test WhatsApp campaign broadcast in sandbox mode

PHASE 6 Security Hardening, Testing & Production Deploy

· *Week 12*

Goals for this phase

- Implement AES-256 field-level encryption for all child PII columns
- Harden the audit log — confirm it is append-only and cannot be deleted by any role
- Implement DPDP Act (India) consent tracking — consent required before child record creation
- Write end-to-end tests covering the full P0 path: assessment submission → Domain XII Yes → alert sent
- Create production Docker builds (multi-stage, minimal image)
- Deploy to Railway or Render — confirm all environment variables, SSL, custom domain
- Create Architecture Decision Records for every architectural decision made during the build

Deliverables — Claude Code must create all files listed below

File / Deliverable	Description	Type
wisdom-backend/app/security/encryption.py	AES-256 field encryption for PII	Python
wisdom-backend/app/security/audit.py	Append-only audit log enforcement	Python
wisdom-backend/app/security/consent.py	DPDP Act consent tracking	Python
wisdom-backend/Dockerfile	Multi-stage production build	Docker
wisdom-frontend/Dockerfile	Multi-stage Next.js production build	Docker
.github/workflows/ci.yml	CI: lint, type-check, test on every PR	GitHub Actions
.github/workflows/deploy.yml	CD: deploy to Railway/Render on main merge	GitHub Actions
tests/test_e2e_p0.py	E2E: DSM-5 submission → Domain XII → alert	pytest
docs/adr/	All architectural decision records from build	Markdown
docs/deployment_runbook.md	Step-by-step production deployment guide	Markdown
docs/security_design.md	Encryption, RBAC+ABAC, audit, consent	Markdown

🔴 CHECKPOINT — Stop and confirm before continuing

- ☐ Run full test suite — all tests pass
- ☐ Confirm child name and DOB fields are encrypted in PostgreSQL (show raw DB value)
- ☐ Demonstrate P0 path: submit DSM-5 with Domain XII = Yes → risk alert logged + WhatsApp sent
- ☐ Show me the deployed application at its production URL with SSL
- ☐ Show me the audit log entry for a child record access
- ☐ CI pipeline passes on a test PR

8. Quick-Copy Phase Prompt Snippets

Paste each block at the start of the corresponding phase. The Global System Context (Section 6) must already be in the conversation before using any of these.

Phase 1 Prompt

```
Begin Phase 1: Project Scaffold, Database & Observability.

1. Initialise wisdom-backend/ as a FastAPI app with the domain folder structure
   from the master prompt (cases, assessments, scheduling, messaging, payments,
   ai, analytics, public, donations, auth, admin).
2. Initialise wisdom-frontend/ as a Next.js 14 App Router project with
   Tailwind CSS v4, shadcn/ui, Framer Motion, and Sentry.
3. Create all SQLAlchemy models as specified.
4. Create docker-compose.yml with PostgreSQL 16 and Redis 7.
5. Create .env.example with ALL environment variables documented.
6. Create and run Alembic migration 001_initial.
7. CRITICAL: Install and initialise Sentry in BOTH repos in Phase 1.
   Send a test event and confirm it appears in the Sentry dashboard.
8. Seed: all Ark Centre rooms (exact names from the Rooms PDF).
9. Seed: full DSM-5 Level 1 (25 questions, 12 domains, scoring thresholds,
   Domain XII suicidal ideation P0 flag).

When complete: run tree wisdom-backend/app && alembic current
Show me the output. Then STOP and wait for my approval.
```

Phase 2 Prompt

```
Begin Phase 2: Authentication, RBAC+ABAC & Admin Panel.

1. Implement JWT auth (access + refresh tokens, httpOnly cookies).
2. Implement Casbin policy engine with the 7 roles from the master prompt.
3. Implement ABAC attribute rules (user must be assigned to a child record
   to access it, unless role is Admin or Chief Therapist).
4. Create FastAPI route guards using Casbin as a FastAPI Depends().
5. Create separate auth flows for staff login and parent/caretaker login.
6. Build the complete Admin Panel at /admin in Next.js with all 8 sections:
   User Management, Role Editor, ABAC Attribute Manager, Consent Tracker,
   Audit Log Viewer, Session Monitor, System Config, Assessment Library.
7. Enforce route guards in wisdom-frontend/middleware.ts.
8. Write pytest tests for all role x resource x ABAC combinations.
9. Output docs/rbac_matrix.md.

When complete: run pytest tests/test_auth.py -v
Show me the full output and the admin panel in the browser. Then STOP.
```

Phases 3–6 Template

```
Begin Phase [N]: [Title].

Refer to the deliverables table in the master prompt for this phase.
Create every file listed. Do not skip any.

Additional instruction for Phase 3 only:
  Use the attached DSM-5 PDF as the actual input to the PDF→Survey pipeline.
  Do not mock this. Parse the real document.

When all deliverables are complete:
  1. List every file you created with its path.
  2. Show me the checkpoint checklist with each item confirmed.
  3. STOP and wait for my approval before starting the next phase.
```

9. Information Still Needed Before Production

Claude Code will ask for these at the relevant phase. Collect them in advance where possible.

Item	Why Needed	Needed By Phase
Meta WhatsApp Business Account ID + token	*Messaging integration*	Phase 4
WhatsApp pre-approved message templates	*Reminder content*	Phase 4
Razorpay API key + webhook secret	*Payment processing*	Phase 4
Stripe API key (for international donors)	*Foreign donations*	Phase 4
LiteLLM preferred model (GPT-4o / Claude / Gemini)	*AI feature prompts*	Phase 3
Sentry DSN for both repos	*Observability Day 1*	Phase 1
Supabase project URL + keys	*Database hosting*	Phase 1
Upstash Redis URL + token	*Cache hosting*	Phase 1
Railway / Render account	*Production hosting*	Phase 6
Custom domain name	*SSL certs, CORS config*	Phase 6
DPDP consent language (legal review)	*India data privacy compliance*	Phase 6
Super Admin email + initial password	*First admin account*	Phase 2
Additional assessments beyond DSM-5 L1	*Assessment library growth*	Phase 3+
Therapist fee structure	*Invoice line items*	Phase 4
Crowdfunding campaign descriptions	*Donation portal content*	Phase 4